```
# ============================================================
# Stock Price Prediction using SVM
# Vineesh Nandi
# ============================================================
# HOW TO USE:
# 1. Go to https://colab.research.google.com
# 2. Create a New Notebook
# 3. Paste this entire code into a cell and run it (Shift+Enter)
# 4. It downloads real historical stock data (Apple Inc.) and
#    trains an SVM regression model, printing real metrics.
# ============================================================

# Install yfinance (Colab usually has it, this ensures it's there)
!pip install yfinance --quiet

import yfinance as yf
import pandas as pd
import numpy as np
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# ------------------------------------------------------------
# STEP 1: Download real historical stock data
# Using Apple (AAPL) - 5 years of daily OHLCV data
# ------------------------------------------------------------
ticker = "AAPL"
df = yf.download(ticker, period="5y", interval="1d", progress=False)
df = df.reset_index()

print(f"Dataset loaded: {df.shape[0]} rows of {ticker} stock data")
print(df.head())

# ------------------------------------------------------------
# STEP 2: Feature Engineering
# Using OHLCV (Open, High, Low, Close, Volume) features
# Target: next day's closing price
# ------------------------------------------------------------
df['MA5'] = df['Close'].rolling(window=5).mean()     # 5-day moving average
df['MA10'] = df['Close'].rolling(window=10).mean()    # 10-day moving average
df['Volatility'] = df['Close'].rolling(window=5).std()  # 5-day rolling volatilit
```

```
df['Target'] = df['Close'].shift(-1)  # next day's close price (what we predict)

df = df.dropna()

features = ['Open', 'High', 'Low', 'Close', 'Volume', 'MA5', 'MA10', 'Volatility'
X = df[features]
y = df['Target']

print(f"\nAfter feature engineering: {X.shape[0]} samples, {X.shape[1]} features"

# ------------------------------------------------------------
# STEP 3: Train/Test Split
# Note: for time series, we don't shuffle - train on past, test on future
# ------------------------------------------------------------
split_idx = int(len(X) * 0.8)
X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]

scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)
y_train_scaled = scaler_y.fit_transform(y_train.values.reshape(-1, 1)).ravel()

print(f"\nTrain set: {X_train.shape[0]} samples (earlier dates)")
print(f"Test set: {X_test.shape[0]} samples (most recent dates)")

# ------------------------------------------------------------
# STEP 4: Hyperparameter Tuning with K-Fold Cross-Validation
# (reduces overfitting, as mentioned in your project description)
# ------------------------------------------------------------
param_grid = {
    'C': [1, 10, 100],
    'gamma': ['scale', 0.01, 0.1],
    'kernel': ['rbf']
}

print("\nRunning GridSearchCV with 5-fold cross-validation (this may take a minut
grid_search = GridSearchCV(SVR(), param_grid, cv=5, scoring='neg_mean_squared_err
grid_search.fit(X_train_scaled, y_train_scaled)

print(f"Best parameters found: {grid_search.best_params_}")

best_model = grid_search.best_estimator_

# ------------------------------------------------------------
```

```
# STEP 5: Evaluate
# ------------------------------------------------------------
y_pred_scaled = best_model.predict(X_test_scaled)
y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
mape = np.mean(np.abs((y_test.values - y_pred) / y_test.values)) * 100

print("\n" + "="*50)
print("RESULTS - copy these numbers into your resume")
print("="*50)
print(f"R² Score: {r2:.4f}")
print(f"RMSE: ${rmse:.2f}")
print(f"MAE: ${mae:.2f}")
print(f"MAPE: {mape:.2f}%")

# ------------------------------------------------------------
# STEP 6: Visualize predictions vs actual (for your README/portfolio)
# ------------------------------------------------------------
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5))
plt.plot(df['Date'].iloc[split_idx:].values, y_test.values, label='Actual Price',
plt.plot(df['Date'].iloc[split_idx:].values, y_pred, label='Predicted Price', col
plt.title(f'{ticker} Stock Price Prediction (SVM) - Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('stock_prediction_plot.png')
plt.show()
print("\nChart saved as stock_prediction_plot.png - download this for your README

# ------------------------------------------------------------
# STEP 7: Save model
# ------------------------------------------------------------
import joblib
joblib.dump(best_model, 'stock_svm_model.pkl')
joblib.dump(scaler_X, 'scaler_X.pkl')
joblib.dump(scaler_y, 'scaler_y.pkl')
print("Model saved as stock_svm_model.pkl")
```